Hotel Nodora · Guía para la defensa

Caso integrador de Programación Orientada a Objetos. Equipo: De Anda Gomez Santiago, Martin Gomez Jorge, Martin Valencia Oscar Jesus y Yanez Sanchez Eduardo.

Cómo usar esta guía: léela con NetBeans abierto al lado. Cada vez que veas el nombre de un método, búscalo en el código (Ctrl+F) y míralo. Al final hay preguntas tipo profe: tápate la respuesta e intenta contestar tú primero, en voz alta.

1. ¿Qué hace el programa? (la respuesta de 30 segundos)

Es un sistema de consola para manejar las reservaciones del Hotel Nodora. Al iniciar, pregunta quién lo va a usar, y cada persona tiene su propio menú:

Actor	Qué puede hacer	Método del menú
Recepcionista el día a día	Ver cuartos libres por tipo, consultar precios y el descuento de socios, calcular el precio total para un cliente, ver el dibujo de un cuarto, hacer y eliminar reservaciones, y ver reservaciones y socios.	`menuRecepcionista()`
Administrador el dinero	Cambiar el precio de un tipo de cuarto, cambiar el descuento de los socios y calcular las ganancias de un mes.	`menuAdministrador()`
Cliente lo suyo	Registrarse como socio (cliente habitual), reservar un cuarto y anular su propia reservación.	`menuCliente()`

El programa arranca con datos de ejemplo (9 cuartos, 2 socios y 3 reservaciones) que se cargan en `cargarDatosDeEjemplo()`. No se guardan en un archivo, así que al cerrar el programa se pierde lo que se haya agregado.

2. Mapa de clases

Clase	Tipo	Para qué sirve
`Cuarto`	abstracta (padre)	Lo que tienen todos los cuartos: `numero` y `precio`, más `mostrarInfo()`. Declara 3 métodos abstractos: `getTipo()`, `getCapacidad()` y `dibujar()`.
`CuartoSimple` `CuartoDoble` `CuartoMatrimonial`	hijas de Cuarto	Cada una responde distinto a los métodos abstractos. Capacidad: Simple 1, Matrimonial 2 y Doble 4. Cada una tiene su propio dibujo.
`Cliente`	abstracta (padre)	Lo que tienen todos los clientes: `curp` y `nombre`. Tiene `aplicarDescuento()` y declara `getTipo()` y `getDescuento()` como abstractos.
`ClienteHabitual`	hija de Cliente	El socio. Agrega `telefono` y el descuento `static` (10% al inicio).
`ClienteEsporadico`	hija de Cliente	El que reserva sin ser socio. Su `getDescuento()` regresa 0.
`Reservacion`	normal	Une un cliente con un cuarto, más la fecha de llegada, las noches y los huéspedes. Guarda el precio por noche ya con descuento.
`Hotel`	normal	El "cerebro": guarda las listas y tiene un método por cada operación.
`Hotel_Nodora`	principal	Tiene el `main`, los menús y los métodos que leen del teclado.

Qué guarda la clase Hotel

```
ArrayList<Cuarto>            cuartos             // todos los cuartos, de cualquier
tipo
HashMap<String, Cliente>     clientesHabituales  // clave = CURP, valor = el socio
ArrayList<Reservacion>       reservaciones       // las reservaciones que existen
```

Idea clave: los menús (`Hotel_Nodora`) piden datos y el objeto `hotel` hace el trabajo. Si te preguntan "¿dónde se hace X?", casi siempre la respuesta es un método de `Hotel`.

3. Conceptos de POO y dónde están en el código

Clase abstracta

`Cuarto` y `Cliente` son `abstract`: no se puede hacer `new Cuarto(...)`. Tiene sentido, porque no existe un cuarto "sin tipo". Siempre es simple, doble o matrimonial. Es la misma idea que `FigurasGeometricas` en clase.

```
public abstract class Cuarto {
    public abstract int getCapacidad();   // sin cuerpo: cada hija pone el suyo
}
```

Herencia: `extends` y `super`

`CuartoSimple extends Cuarto` significa que ya trae `numero`, `precio`, `mostrarInfo()` y los getters sin escribirlos otra vez. En el constructor, `super(numero, precio)` le pasa esos datos al constructor del padre.

```
public CuartoSimple(int numero, double precio){
    super(numero, precio);   // los guarda la clase Cuarto
}
```

Sobrescritura: `@Override`

Cada hija escribe su propia versión de los métodos abstractos. `ClienteHabitual` también sobrescribe `mostrarInfo()`: primero llama a `super.mostrarInfo()` (nombre, CURP y tipo) y después agrega su teléfono y su descuento.

Polimorfismo (el concepto más importante del proyecto)

Una variable de tipo padre puede guardar cualquier hija, y al llamar a un método se ejecuta **la versión de la hija real**. Está en tres lugares:

- `ArrayList<Cuarto>` guarda simples, dobles y matrimoniales en la misma lista.
- `c.dibujar()` en `Hotel.dibujarTipo()` hace un dibujo distinto según el tipo de cuarto.
- `cliente.aplicarDescuento(precio)` llama a `getDescuento()`. A un socio le quita el 10% y a un esporádico le quita 0, **sin ningún** `if` que pregunte el tipo.

Encapsulación: `private` y `protected`

- `protected` en `Cuarto` y `Cliente`: las hijas usan los atributos directo, pero el resto del programa no.
- `private` en `Reservacion` y `Hotel`: desde fuera solo se pueden leer con getters.
- `Cliente` no tiene setters, porque la CURP y el nombre no se cambian.

`static`

```
private static int porcentajeDescuento = 10;   // en ClienteHabitual
```

Es **uno solo para todos los socios**. Si el administrador lo cambia, les cambia a todos al mismo tiempo. Los métodos `getPorcentajeDescuento()` y `setPorcentajeDescuento()` también son static, así que se llaman con el nombre de la clase (`ClienteHabitual.getPorcentajeDescuento()`) aunque no haya ningún socio. El `Scanner sn` de `Hotel_Nodora` también es static para que lo compartan todos los métodos.

Sobrecarga

Hay dos métodos `leerEntero` con el mismo nombre y diferentes parámetros: `leerEntero(mensaje)` y `leerEntero(mensaje, min, max)`. El segundo usa al primero y además revisa el rango.

ArrayList y HashMap

- **ArrayList** se usa para cuartos y reservaciones. Para buscar se recorre con un `for` (ver `buscarCuarto()`).
- **HashMap** se usa para los socios, con la CURP como clave. `get(curp)` encuentra al socio directo, sin recorrer nada, y regresa `null` si no existe. `containsKey()` evita que se registre la misma CURP dos veces.

Validación con try/catch y ciclos

`leerEntero()` lee la línea completa y la convierte con `Integer.parseInt`. Si escriben letras, se lanza `NumberFormatException`, el `catch` la atrapa y el `do-while` vuelve a preguntar. Todos los menús usan `do-while` con `switch`, porque el menú tiene que mostrarse al menos una vez.

4. Recorrido de una reservación, paso a paso

Método `hacerReservacion(hotel)`. Lo usan el recepcionista y el cliente, por eso está en un solo lugar y no repetido en los dos menús.

1. Muestra todos los cuartos libres con `hotel.listarTodosDisponibles()`.
2. Pide el número de cuarto y lo busca con `buscarCuarto()`. Si es `null`, no existe y se sale con `return`.
3. Revisa que esté libre con `estaDisponible()`. Un cuarto está libre si `buscarReservacion()` regresa `null`.
4. Pide la CURP y la busca en el HashMap. Si la encuentra, es socio y tiene descuento. Si no, pide el nombre y crea un `ClienteEsporadico`.
5. Pide los huéspedes, con un máximo igual a `cuarto.getCapacidad()` (polimorfismo otra vez), la fecha y las noches.
6. Crea la `Reservacion`. En su constructor se calcula y se guarda `precioPorNoche = cliente.aplicarDescuento(cuarto.getPrecio())`.
7. La agrega a la lista con `hotel.agregarReservacion()` y muestra el resumen.

5. Los cálculos, con ejemplos del propio programa

Descuento

```
precio - precio * getDescuento() / 100
Cuarto simple $800 para un socio (10%):   800 - 800*10/100 = $720 por noche
```

Total de una reservación

```
precioPorNoche * noches       // calcularTotal()
```

Ganancias de un mes (todos los meses tienen 30 días)

`Reservacion.nochesEnMes(mes, anio)` recorre la estancia noche por noche, como si pasara las hojas de un calendario: suma 1 al día, si pasa de 30 regresa al 1 y avanza el mes, y si el mes pasa de 12 avanza el año. Solo cuenta las noches que caen en el mes que se pidió. Después, `Hotel.calcularGanancias()` multiplica esas noches por el precio por noche de cada reservación y suma todo.

Reservación de ejemplo	Precio/noche	Octubre 2026	Noviembre 2026
Ana (socia), cuarto 101 Simple, llega 28/10, 5 noches	$720	3 noches (28, 29, 30) = $2,160	2 noches (1, 2) = $1,440
Luis (esporádico), cuarto 201 Doble, llega 10/11, 3 noches	$1,500	0	3 noches = $4,500
Carlos (socio), cuarto 302 Matrimonial, llega 15/11, 2 noches	$1,080	0	2 noches = $2,160
Ganancias del mes		**$2,160**	**$8,100**

Para la demo: entra como Administrador, opción 3, mes 11, año 2026. Debe salir **$8,100.00**. Con el mes 10 debe salir **$2,160.00**. Es una buena forma de demostrar que la reservación de Ana se reparte entre dos meses.

6. Decisiones que tomó el equipo (y por qué)

Decisión	Por qué
"Registrarse" = volverse socio (cliente habitual)	El que se registra queda guardado en el HashMap y recibe descuento. Cualquiera que reserve sin registrarse es esporádico.
Un cuarto tiene máximo una reservación	Para simplificar y no tener que comparar rangos de fechas. Si el cuarto está reservado, ya no aparece como disponible.
Todos los meses tienen 30 días	Lo indica el enunciado. Por eso el día va de 1 a 30.
La reservación guarda su propio precio por noche	Si el administrador cambia precios o el descuento después, las reservaciones que ya estaban hechas conservan su precio.
El precio vive en cada cuarto, no en el tipo	Para cambiar el precio de un tipo, `cambiarPrecioDelTipo()` recorre todos los cuartos y se lo cambia a los de ese tipo.
El descuento es `static`	Es igual para todos los socios.
La CURP se valida solo por largo (18 caracteres)	Para no complicar la validación. Se pasa a mayúsculas para aceptarla aunque la escriban en minúsculas.

7. Limitaciones conocidas (mejor decirlas tú antes de que te las encuentren)

- No hay archivo ni base de datos, así que todo se pierde al cerrar el programa.
- No se puede reservar un cuarto para otra fecha si ya tiene una reservación.
- No revisa si la fecha de llegada ya pasó.
- El nuevo precio solo acepta números enteros, porque se lee con `leerEntero`.
- Al registrarse como socio, el nombre y el teléfono pueden quedar vacíos.
- La CURP solo se revisa por largo, no por formato.

Si te preguntan "¿cómo lo mejorarían?", cualquiera de estos puntos es una buena respuesta. Por ejemplo: "guardaríamos los datos en un archivo" o "permitiríamos varias reservaciones por cuarto si las fechas no se enciman".

8. Preguntas tipo profe (tápate la respuesta)

1. ¿Por qué Cuarto tiene clases hijas en vez de ser una sola clase?

Porque todos los cuartos comparten número y precio, pero cada tipo tiene distinta capacidad y distinto dibujo. Lo común va en el padre y lo diferente en cada hija. Además, para agregar un tipo nuevo (una Suite) solo se crea otra clase hija.

2. ¿Por qué Cuarto es abstracta?

Porque no existe un cuarto sin tipo, así que no tiene sentido hacer `new Cuarto()`. Además obliga a cada hija a escribir `getTipo()`, `getCapacidad()` y `dibujar()`.

3. ¿Dónde usan polimorfismo?

En `aplicarDescuento()`: llama a `getDescuento()` y cada tipo de cliente da el suyo (10 o 0) sin usar un `if`. También en `dibujar()` y `getCapacidad()`, y en el `ArrayList<Cuarto>` que guarda los 3 tipos.

4. ¿Por qué el descuento es static?

Porque es el mismo para todos los socios. Si fuera un atributo normal, habría que cambiárselo a cada socio uno por uno. Además se puede consultar con el nombre de la clase aunque no haya socios.

5. ¿Por qué usaron HashMap para los socios y ArrayList para lo demás?

A los socios siempre se les busca por CURP, que es única. Con el HashMap se encuentran directo con `get(curp)`. Los cuartos y las reservaciones se recorren completos para listarlos, así que una lista basta.

6. ¿Qué pasa si en el menú escribo "hola" en vez de un número?

`Integer.parseInt` lanza `NumberFormatException`, el `catch` la atrapa, muestra "debes teclear un número entero" y el `do-while` lo vuelve a pedir. El programa no truena.

7. ¿Para qué sirve super en el constructor de CuartoSimple?

Le manda el número y el precio al constructor de `Cuarto`, que es donde están declarados esos atributos.

8. Si el administrador sube el precio, ¿cambia lo que paga alguien que ya reservó?

No. La reservación guarda su `precioPorNoche` en el momento en que se crea, así que el cambio solo afecta a reservaciones nuevas.

9. Explíquenme cómo calculan las ganancias de un mes.

Por cada reservación se cuentan las noches que caen en ese mes con `nochesEnMes()`, que avanza día por día con meses de 30 días. Esas noches se multiplican por el precio por noche y se suman. Ejemplo: Ana llega el 28 de octubre por 5 noches, así que cuenta 3 noches en octubre y 2 en noviembre.

10. ¿Cómo evitan que un cliente cancele la reservación de otro?

`anularReservacion(curp, numeroCuarto)` solo borra la reservación si la CURP de esa reservación es igual a la CURP que se tecleó.

11. ¿Qué es la sobrecarga y dónde la usaron?

Son métodos con el mismo nombre y diferentes parámetros: `leerEntero(mensaje)` y `leerEntero(mensaje, min, max)`.

12. ¿Qué diferencia hay entre private y protected aquí?

`protected` (en `Cuarto` y `Cliente`) deja que las hijas usen los atributos directo. `private` (en `Reservacion`) solo deja usarlos dentro de esa clase; desde fuera se leen con getters.

13. ¿Cómo agregarían un tipo de cuarto Suite?

Se crea `class Suite extends Cuarto` con sus 3 métodos `@Override`, se agrega la opción en `elegirTipo()` y se agregan sus cuartos en `cargarDatosDeEjemplo()`. Lo demás funciona igual gracias al polimorfismo.

14. ¿Por qué usan do-while en los menús y no while?

Porque el menú tiene que mostrarse por lo menos una vez antes de revisar si el usuario eligió salir.

15. ¿Por qué leen con nextLine() y parseInt en vez de nextInt()?

Porque `nextInt()` deja el salto de línea en el Scanner y el siguiente `nextLine()` lo lee vacío. Leyendo la línea completa ese problema no pasa.

9. Acordeón rápido

Si preguntan por…	Di esto y señala…
Abstracta	No existe un cuarto sin tipo → `Cuarto`, `Cliente`
Herencia	Lo común en el padre → `extends`, `super(...)`
Polimorfismo	Misma llamada, distinta respuesta → `aplicarDescuento()`, `dibujar()`
Encapsulación	Atributos protegidos y getters → `private` / `protected`
static	Uno para todos los socios → `porcentajeDescuento`
Sobrecarga	Mismo nombre, otros parámetros → `leerEntero`
Colecciones	Socios por CURP → `HashMap`; el resto → `ArrayList`
Excepciones	Letras en vez de número → `try/catch NumberFormatException`